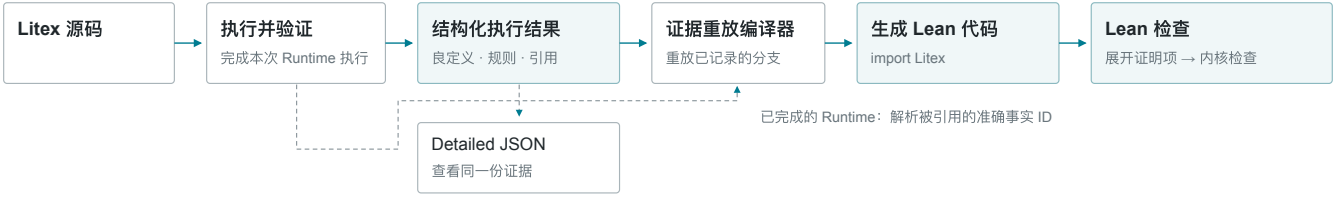
从已验证的证据，到 Lean 证明

保留原始数学对象、构造条件，以及 Litex 实际选中的证明路线。



JSON 是结果的可读展示；当前编译器直接消费结构化结果与 Runtime。

a 读出已检查的依据

真实源码与 Detailed 字段

```
forall a C:
    a + 0 = a
```

引入一个复数成员

a ∈ ℂ，记录为 f15

检查加法的构造条件

a ∈ ℂ、0 ∈ ℂ 使 a + 0 合法

实际选中的等式证明

```
"searched_proof": {
  "type": "builtin_rule",
  "rule": "Calculation",
  "mode": "rational" }
```

局部 f1 → 全称 f2

b 重放选中的证明路线

已记录的证据 → 实际生成的代码

成员关系引用 f15

```
h_15 : Litex.In … Litex.C
```

加法的良定义证据

```
Litex.add … h_15
  (Litex.numberInC …)
```

Calculation / rational

```
Litex.NativeBridge.
  sameOfDenoteNumber …
  (by ring!)
```

存储 f1 / 导出 f2

```
have h_1 … / theorem h_2 …
```

在选中的适配分支内，用固定归一化检查。
编译器沿着记录的路线生成证明。

c Litex.lean 提供什么

Lean 中的数学含义与已证明的桥接

1 对象保留构造条件

Obj α = 对象值 + WD 良定义证据

```
Representation M α · denote
```

2 给事实明确的数学含义

In 表示成员关系；Same 表示含义相等

```
Semantics · In · Same
```

3 桥接规律本身也由 Lean 证明

数值等式 + 两端的含义证据 → Same

```
NativeBridge.sameOfDenoteNumber
```

解释参数 M : Semantics 显式保留。
生成定理以该解释满足的规律为条件。

import Litex 引入这些定义和已证明的规律。

已证明的语义库 + 生成的证明项 → Lean 独立检查的定理

可选下一步：手写 Adapter → 普通 Lean / Mathlib 接口。